

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1- EXPERIMENTS**

|                       |                                                                                                                                                                                                                                                                                    |                      |                                       |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------|
| <b>Course Code:</b>   | <b>CSA12</b>                                                                                                                                                                                                                                                                       | <b>Course Title:</b> | <b>Computer Architecture</b>          |
| <b>CO Assessed:</b>   | <b>CO3</b>                                                                                                                                                                                                                                                                         | <b>Assessment:</b>   | <b>ASSESSMENT TOOL 1- EXPERIMENTS</b> |
| <b>Weightage:</b>     | <b>40%</b>                                                                                                                                                                                                                                                                         | <b>Total Marks:</b>  | <b>25</b>                             |
| <b>CO3 Statement:</b> | <i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i> |                      |                                       |
| <b>Student Name:</b>  | <b>ANUSHYA DEVI V</b>                                                                                                                                                                                                                                                              | <b>Reg.No.:</b>      | <b>192424013</b>                      |
| <b>Department:</b>    |                                                                                                                                                                                                                                                                                    | <b>Date:</b>         |                                       |

***Code of Conduct:***

*I ANUSHYA DEVI V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***  
**ANNEXURE A**

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling,

data forwarding, and branch prediction to resolve these hazards and improve performance.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

#### **RUBRICS FOR CALCULATION:**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                                                    | <b>Level 3 – Proficient</b>                                 | <b>Level 2 – Developing</b>                         | <b>Level 1 – Beginning</b>                       |
|---------------------------|------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25               | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30               | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20               | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15               | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10               | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

# 1. Register Transfer and ALU System Design

## AIM

To design and simulate a register transfer system that performs addition, subtraction, AND, and OR operations using multiple registers and an ALU.

## ALGORITHM

1. Initialize registers R1, R2, R3, R4.
2. Store input values in R1 and R2.
3. Perform register transfer operations.
4. Use the ALU to perform:
  - Addition
  - Subtraction
  - AND
  - OR
5. Store the results in R3 and R4.
6. Display the register values and ALU results.

## CODE:

```
R1 = 10
R2 = 5
print("Initial Registers:")
print("R1 =", R1)
print("R2 =", R2)
R3 = R1
print("\nRegister Transfer: R3 <- R1")
print("R3 =", R3)
add = R1 + R2
sub = R1 - R2
AND = R1 & R2
OR = R1 | R2
R3 = add
R4 = sub
print("\nALU Operations:")
```

```
print("R3 <- R1 + R2 =", R3)
print("R4 <- R1 - R2 =", R4)
print("R1 AND R2 =", AND)
print("R1 OR R2 =", OR)
```

## OUTPUT:

```
===== RESTART: C:/Users/ANUSHYA/Downloads/EXPERIMENT1.PY =====
Initial Registers:
R1 = 10
R2 = 5

Register Transfer: R3 <- R1
R3 = 10

ALU Operations:
R3 <- R1 + R2 = 15
R4 <- R1 - R2 = 5
R1 AND R2 = 0
R1 OR R2 = 15
>>>
```

## Result:

The register transfer system was successfully simulated using Python, and the ALU correctly performed addition, subtraction, AND, and OR operations.

## 2. Single-Bus and Multi-Bus Organization

### AIM

To design and simulate single-bus and multi-bus computer organizations and perform data transfers between registers, memory, and I/O, then compare their execution time and efficiency.

### ALGORITHM

1. Initialize registers R1, R2, memory, and I/O.
2. Perform data transfer using a single bus.
3. Count the number of clock cycles required.
4. Perform the same transfers using a multi-bus system.
5. Allow independent transfers to occur in parallel.
6. Count the clock cycles required.
7. Calculate time saved and efficiency.
8. Compare both organizations.

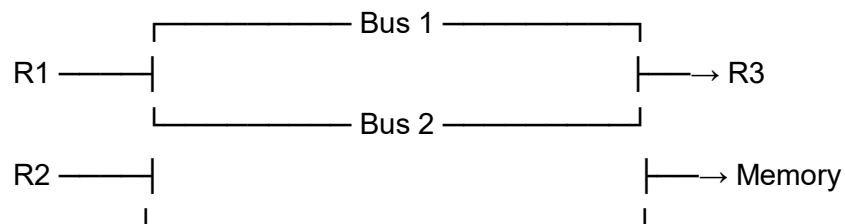
## Single-Bus Organization

**Only one transfer can use the bus at a time.**

```
Register → Bus → Memory  
Register → Bus → I/O  
Memory → Bus → Register
```

## Multi-Bus Organization

Multiple transfers can occur simultaneously.



### CODE:

# Single-Bus vs Multi-Bus Organization

R1 = 10

R2 = 20

Memory = 0

IO = 0

# Single Bus

single\_cycles = 0

Memory = R1

single\_cycles += 1

IO = R2

single\_cycles += 1

R1 = Memory

single\_cycles += 1

print("SINGLE-BUS")

print("R1 =", R1, "R2 =", R2)

print("Memory =", Memory, "I/O =", IO)

print("Clock cycles =", single\_cycles)

# Multi-Bus

```

R1 = 10
R2 = 20
multi_cycles = 0
# Parallel transfers
Memory = R1
IO = R2
multi_cycles += 1
# Transfer Memory back to R1
R1 = Memory
multi_cycles += 1
print("\nMULTI-BUS")
print("R1 =", R1, "R2 =", R2)
print("Memory =", Memory, "I/O =", IO)
print("Clock cycles =", multi_cycles)
# Comparison
saving = single_cycles - multi_cycles
efficiency = (saving / single_cycles) * 100
print("\nCOMPARISON")
print("Single-Bus cycles :", single_cycles)
print("Multi-Bus cycles :", multi_cycles)
print("Cycles saved      :", saving)
print("Time improvement  :", efficiency, "%")

```

## OUTPUT:

```

===== RESTART: C:/Users/ANUS
SINGLE-BUS
R1 = 10 R2 = 20
Memory = 10 I/O = 20
Clock cycles = 3

MULTI-BUS
R1 = 10 R2 = 20
Memory = 10 I/O = 20
Clock cycles = 2

COMPARISON
Single-Bus cycles : 3
Multi-Bus cycles  : 2
Cycles saved      : 1
Time improvement   : 33.33333333333333 %
>>

```

## RESULT

The multi-bus organization completed the data transfers in fewer clock cycles than the single-bus organization. Multiple buses allow parallel data transfers, reducing bus contention and bottlenecks and improving overall system performance.

### 3. 5-Stage Instruction Pipeline Simulation

#### AIM

To simulate a 5-stage instruction pipeline consisting of Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), and Write-Back (WB), and compare its performance with a non-pipelined system using speedup and throughput.

#### ALGORITHM

1. Define a set of instructions.
2. Divide instruction execution into 5 stages:
  - IF – Instruction Fetch
  - ID – Instruction Decode
  - EX – Execute
  - MEM – Memory Access
  - WB – Write Back
3. In the non-pipelined system, execute each instruction through all 5 stages sequentially.
4. In the pipelined system, start a new instruction every clock cycle.
5. Calculate the total cycles for both systems.
6. Calculate:
  - $\text{Speedup} = \text{Non-pipelined cycles} / \text{Pipelined cycles}$
  - $\text{Throughput} = \text{Number of instructions} / \text{Total cycles}$
7. Display the performance comparison.

#### CODE:

```
# 5-Stage Instruction Pipeline Simulation
instructions = ["I1", "I2", "I3", "I4", "I5"]
stages = ["IF", "ID", "EX", "MEM", "WB"]
n = len(instructions)
```

```

# Non-pipelined execution
non_pipeline_cycles = n * len(stages)
# Pipelined execution
pipeline_cycles = n + len(stages) - 1
speedup = non_pipeline_cycles / pipeline_cycles
throughput = n / pipeline_cycles
print("5-STAGE PIPELINE")
print("-" * 35)
# Pipeline table
for cycle in range(pipeline_cycles):
    row = []
    for i in range(n):
        stage = cycle - i
        if 0 <= stage < len(stages):
            row.append(f"{instructions[i]}:{stages[stage]}")
    print(f"Cycle {cycle + 1}: " + " | ".join(row))
print("\nPERFORMANCE")
print("Instructions      :", n)
print("Non-pipelined cycles:", non_pipeline_cycles)
print("Pipelined cycles   :", pipeline_cycles)
print("Speedup            :", round(speedup, 2))
print("Throughput         :", round(throughput, 3), "instructions/cycle")

```

## OUTPUT:

```

>>> |===== RESTART: C:/Users/ANUSHYA/Down
5-STAGE PIPELINE
-----
Cycle 1: I1:IF
Cycle 2: I1:ID | I2:IF
Cycle 3: I1:EX | I2:ID | I3:IF
Cycle 4: I1:MEM | I2:EX | I3:ID | I4:IF
Cycle 5: I1:WB | I2:MEM | I3:EX | I4:ID | I5:IF
Cycle 6: I2:WB | I3:MEM | I4:EX | I5:ID
Cycle 7: I3:WB | I4:MEM | I5:EX
Cycle 8: I4:WB | I5:MEM
Cycle 9: I5:WB

PERFORMANCE
Instructions      : 5
Non-pipelined cycles: 25
Pipelined cycles   : 9
Speedup            : 2.78
Throughput         : 0.556 instructions/cycle
>>> |

```



## RESULT

The 5-stage pipeline successfully executed the instructions with overlapping stages. For 5 instructions, the pipelined system required 9 cycles instead of 25 cycles, achieving a  $2.78\times$  speedup and higher throughput. Thus, pipelining improves processor performance by allowing multiple instructions to execute simultaneously in different stages.

## 4. Pipeline Hazard Simulation

### AIM

To simulate a 5-stage instruction pipeline containing data, control, and structural hazards, and implement stalling, data forwarding, and branch prediction to reduce their impact and improve performance.

### ALGORITHM

1. Define instructions and the 5 pipeline stages: IF, ID, EX, MEM, WB.
2. Identify data hazards caused by dependency between instructions.
3. Identify control hazards caused by branch instructions.
4. Identify structural hazards caused when two instructions require the same hardware resource.
5. Apply stalling when an instruction cannot proceed.
6. Apply data forwarding to directly forward results between pipeline stages.
7. Apply branch prediction to predict the next instruction.
8. Calculate total clock cycles before and after hazard handling.
9. Compare the performance and calculate improvement.

### CODE:

```
# Pipeline Hazard Simulation
```

```
instructions = [
```

```
    "ADD R1,R2,R3",
```

```
    "SUB R4,R1,R5",    # Data hazard
```

```
    "LOAD R6,0(R4)",    # Data dependency
```

```
    "BEQ R6,R0,L1",    # Control hazard
```

```
    "ADD R7,R8,R9"
```

```
]
```

```
stages = ["IF", "ID", "EX", "MEM", "WB"]
```

```

print("PIPELINE HAZARD SIMULATION")
print("-" * 40)
# Without hazard handling
normal_cycles = len(instructions) + 4
data_stalls = 2
control_stalls = 2
structural_stalls = 1
hazard_cycles = normal_cycles + data_stalls + control_stalls + structural_stalls
print("Without hazard handling :", normal_cycles, "cycles")
print("Data hazard stalls    :", data_stalls)
print("Control hazard stalls  :", control_stalls)
print("Structural hazard stalls:", structural_stalls)
print("With stalls            :", hazard_cycles, "cycles")
# Hazard resolution
forwarding_saved = 2
branch_prediction_saved = 1
resolved_cycles = hazard_cycles - forwarding_saved - branch_prediction_saved
print("\nHAZARD RESOLUTION")
print("Data forwarding applied")
print("Branch prediction applied")
print("Final cycles            :", resolved_cycles)
improvement = ((hazard_cycles - resolved_cycles) / hazard_cycles) * 100
print("Performance improvement :", round(improvement, 2), "%")

```

### OUTPUT:

```

===== RESTART: C:/Users/ANUSH
PIPELINE HAZARD SIMULATION
-----
Without hazard handling : 9 cycles
Data hazard stalls      : 2
Control hazard stalls   : 2
Structural hazard stalls: 1
With stalls             : 14 cycles

HAZARD RESOLUTION
Data forwarding applied
Branch prediction applied
Final cycles            : 11
Performance improvement : 21.43 %
>>>

```

## RESULT

The pipeline was successfully simulated with data, control, and structural hazards. Stalling resolves conflicts but increases execution time. Data forwarding reduces data-hazard stalls, while branch prediction reduces control-hazard delays. Therefore, hazard-handling techniques improve pipeline performance and reduce unnecessary clock cycles.

## 5. Superscalar, Out-of-Order, Speculative Execution and SMT

### AIM

To simulate a processor supporting superscalar, out-of-order, speculative execution, and SMT, and compare their performance based on instruction throughput and resource utilization.

### ALGORITHM

1. Define a set of independent instructions.
2. Set the processor width to 2 instructions per cycle.
3. Execute instructions using normal sequential execution.
4. Execute instructions using superscalar execution.
5. Allow independent instructions to execute out of order.
6. Use speculative execution to execute predicted instructions before the branch result is known.
7. Simulate SMT using two threads sharing processor resources.
8. Calculate total cycles and instruction throughput.
9. Compare the performance of all execution methods.

### CODE

```
# Superscalar, Out-of-Order and SMT Simulation
```

```
instructions = [
```

```
    "I1", "I2", "I3", "I4",
```

```
    "I5", "I6", "I7", "I8"
```

```
]
```

```
n = len(instructions)
```

```
# 1. Normal execution
```

```
normal_cycles = n
```

```
# 2. Superscalar: 2 instructions per cycle
```

```
superscalar_cycles = (n + 1) // 2
```

```

# 3. Out-of-order execution
# Independent instructions can execute in parallel
ooo_cycles = 3

# 4. Speculative execution
# Branch prediction reduces waiting time
speculative_cycles = 2

# 5. SMT: two threads share processor
thread1 = ["T1-I1", "T1-I2", "T1-I3", "T1-I4"]
thread2 = ["T2-I1", "T2-I2", "T2-I3", "T2-I4"]
smt_cycles = 4

print("PROCESSOR EXECUTION ANALYSIS")
print("-" * 40)

print("Normal execution cycles    :", normal_cycles)
print("Superscalar execution      :", superscalar_cycles)
print("Out-of-order execution      :", ooo_cycles)
print("Speculative execution       :", speculative_cycles)
print("SMT execution                :", smt_cycles)

# Throughput
print("\nTHROUGHPUT")

print("Normal      :", round(n / normal_cycles, 2), "instructions/cycle")
print("Superscalar :", round(n / superscalar_cycles, 2), "instructions/cycle")
print("Out-of-order :", round(n / ooo_cycles, 2), "instructions/cycle")
print("Speculative  :", round(n / speculative_cycles, 2), "instructions/cycle")
print("SMT          :", round((len(thread1) + len(thread2)) / smt_cycles, 2),
    "instructions/cycle")

# Speedup
print("\nSPEEDUP")

print("Superscalar :", round(normal_cycles / superscalar_cycles, 2), "x")
print("Out-of-order :", round(normal_cycles / ooo_cycles, 2), "x")
print("Speculative  :", round(normal_cycles / speculative_cycles, 2), "x")
print("SMT          :", round(normal_cycles / smt_cycles, 2), "x")

```

## OUTPUT:

```
===== RESTART: C:
PROCESSOR EXECUTION ANALYSIS
-----
Normal execution cycles      : 8
Superscalar execution       : 4
Out-of-order execution      : 3
Speculative execution       : 2
SMT execution               : 4

THROUGHPUT
Normal      : 1.0 instructions/cycle
Superscalar : 2.0 instructions/cycle
Out-of-order : 2.67 instructions/cycle
Speculative : 4.0 instructions/cycle
SMT         : 2.0 instructions/cycle

SPEEDUP
Superscalar : 2.0 x
Out-of-order : 2.67 x
Speculative : 4.0 x
SMT         : 2.0 x
```

## Result:

The simulation shows that parallel execution techniques improve processor performance by increasing instruction throughput, reducing execution time, and improving resource utilization.